

Interface Design: A Neglected Issue in Educational Software

Douglas Frye

Elliot Soloway

Department of Computer Science
Yale University
P.O. Box 2158
New Haven, Connecticut 06520

Abstract

The user interface is particularly important for educational software because 1) it must provide an entry to the content domain of the program rather than vice versa and 2) it must be sensitive to the general skill and/or developmental level of the user. In spite of these special characteristics, interface design for educational software has been given little attention. This study evaluates a representative interface from arithmetic software now used in the schools. It was found that the interface caused students a large number of difficulties. These difficulties were sufficient to interfere with the instructional effectiveness of the software. Designing interfaces that will benefit educational software will require careful study of the users of these programs along with an in-depth understanding of the domains being taught.

Keywords

Interface design, educational software, direct manipulation interfaces

Introduction

Interface design for educational software has been accorded little study. Nonetheless, the interface for this type of software is even more important than usual.

- *With educational software, the interface must provide an entry to the content domain rather than vice versa.*

With other types of software, the user typically has some idea of what the software is meant to do—the first-time user of a word processor will know some of the ways the program should manipulate text—and so may be able to employ that knowledge to decipher the interface. The user

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

©1987 ACM-0-89791-213-6/87/0004/0093 \$00.75

of a piece of educational software will, almost by definition, not have that advantage, since the purpose of the program itself is to teach the domain.

There is a further requirement for an interface to a piece of educational software.

- *It must be sensitive to the user's general knowledge and/or developmental level.*

Given that the interface must introduce the user to the domain and not the reverse, it will need to make provision for variations in the skills of different users.¹ This point is likely to be especially important for educational software designed for children where skill and developmental differences are the rule rather than the exception. If an interface cannot span some of these differences, then it may be of very limited utility in education.

Our research has begun to examine the importance of the interface in educational software. We have started this work by studying how students react to some of the common interfaces present in schools today. This paper reports a study of children's experience with a set of programs designed to teach simple arithmetic. Our interests in this research have been to uncover 1) some of the specific difficulties a representative interface can cause, 2) determine whether these difficulties actually impair the instructional effectiveness of the software tested, and 3) attempt to see if the interface presented different difficulties to children with different general skill and/or developmental levels.

Methods

Software Tested

The arithmetic software we have been testing is a part of a series of commercially available educational computer programs for children.² There is a separate program for

¹It is also likely that different educational domains may require different interfaces. John Anderson and his group's computer-based tutors for LISP (Farrell, Anderson and Reiser, 1984) and geometry (Anderson, Boyle and Yost, 1985) represent two of the most advanced and effective pieces of educational software currently available. In spite of the fact that both are based on the same theory of human cognition, the interfaces for the two tutors turned out to be very different.

counting, addition and subtraction. A detailed description of the responses necessary to operate the addition program and a picture of the initial screen is given in Figures 1 and 2. Basically the program presented a simple addition problem on the screen along with two sets of objects equal to the numbers in the problem. The user had to give the answer to the problem by pressing number keys from the normal keyboard. It was necessary to press the return key to enter the final (possibly multi-digit) answer. If a problem was answered incorrectly three times in a row, the program entered a remedial sequence that illustrated the correct answer. After every problem it was again necessary to use the return key to operate a small menu that allowed the choice between a new problem, the repetition of the previous problem, or exit from the program. The counting and subtraction programs followed very similar design principles.

Subjects

Our subjects in this study were 20 5-year olds and 20 7-year olds from a public primary school. All of the subjects had some exposure to personal computers at school or at home. All volunteered to try out the software we were testing. All were of the recommended ages for this software.

Methodology and Design

The children each tried out the programs individually in a separate room in the school. A typical session lasted approximately an hour. We first demonstrated the programs and then the child was allowed to work through them freely. Each session was videotaped.

Probes. Subjects were asked specific probe questions at set points during the session. The questions were designed to assess whether the child could read the information on the screen, operate the program, understand feedback from the program, keep up with the pace of the program, and know what the point of the program was (please see Figure 3 for a summary of the probe questions).

Prompts. It was often necessary to prompt the children when they had difficulty operating the interface. These prompts became a major part of the results of the study since they revealed where and how often interface difficulties occurred. We employed a series of graded prompts in order to try to elicit the best performance from the children possible. When the child first experienced difficulty,

the very general prompt of "What should you do next?" was used. If the child continued to be unable to operate this part of the program, several more and more specific hints were given until finally the adult just supplied the response that was needed.

After using the software, all of the children were given the Arithmetic Subscale of the Wechsler Intelligence test and two counting tasks to gauge their general arithmetic skills.

The Addition Program

Initial display. The program begins with the presentation of an addition problem in large characters across the top of the screen. Problems are restricted to single-digit numbers and are always shown in the form of ' $x + y$ '. Next, a square box is drawn beneath each number. Each box is filled with as many instances of an object as the number above the box. The question 'How many?' is then written in normal text at the bottom of the screen.

Correct response. The child responds to the program by entering a 1- or 2-digit number using the keyboard. The number entered is displayed next to the 'How many?' question. The return key must then be struck. If the child has given the correct answer, then the problem is replaced by the correct sum, the boxes are dissolved, and all of the objects are grouped together. They then move off the screen one by one while the sum decrements to zero.

Incorrect response. If the child's answer is incorrect, it disappears from the screen and two more chances are given. On the third attempt, an incorrect response causes the computer to beep. The program then draws crossed lines through each of the objects in turn in the two boxes. A number is shown incrementing as each object is marked. The final number, of course, represents the correct sum.

Final display. No matter whether the correct or error sequence was followed, the final frame for the problem always shows a number on top representing the correct sum with that many instances of the object below it. There is also a line across the lower third of the screen. Beneath that line is a picture of the object used in the problem, a picture of another object and a stop sign. Pressing the space bar on the keyboard moves an arrow so that it points to these pictures one after another. Pressing the return key when the arrow is in the proper position results in the same addition problem being presented again, a new problem, or exit from the program.

²The particular software we used in our studies was produced by Compu-Teach, Inc. However, while this software has some unique features (e.g., excellent color graphics), it nonetheless closely resembles that of the popular "Sticky Bear" series of software produced by Xerox, and most other programs designed to teach counting, adding, and subtracting. Thus, we believe that our results have importance beyond the particular make of software we tested. As an interesting aside, the programs we tested have won awards from various parents and software magazines. Needless to say, adults awarded the prizes—not the children who had to use the software!

Figure 1: Summary description of the addition program.

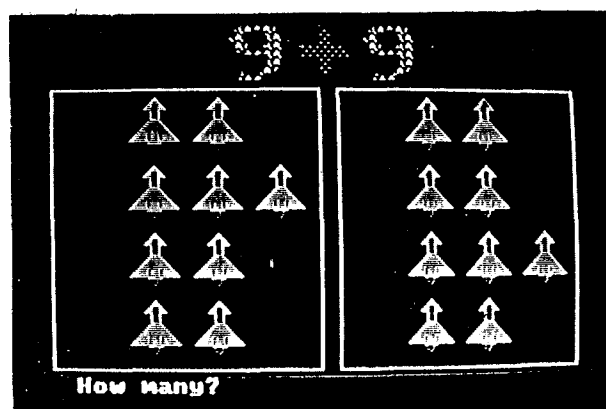


Figure 2: The initial display screen for the addition program.

The Probe Questions

Screen display. A set of questions was asked to determine how well the child took in the information presented on the screen. The questions included whether the child could read the addition problem, read the "How many?" prompt, and recognize the objects used to fill the boxes.

Operation of the program. Probes were done both during and after the child's use of the program to learn how much the child knew of the program's operation. Did the subject know how to enter an answer and move to the next problem. Could they predict what the program would do after they took a certain action.

Feedback. In these questions we wanted to test if the behavior of the program was adequate to inform the subjects that they had gotten a problem correct or not. If not, what exactly was their understanding of the error sequence the program provided.

Pace. At several points in the program the children were asked if they were bored or if they did not like the program any longer. We attempted to find out if there were particular sequences of events in the problems that, because they were too fast or too slow, caused individual children to become distracted or lose interest.

Interpretation of the program. Subjects were asked several questions at the end of the session to learn what purpose they thought the program had; why they thought we were asking them to try it; and whether they thought it was designed to teach them, test them or function as a game.

Figure 3: Summary of the probe questions asked of the children for the arithmetic programs.

Results

The study found, rather surprisingly, that the children experienced many difficulties operating these apparently simple programs. Some of the children's errors were quite obvious. Others constituted what seemed to be very subtle misunderstandings between the children and the programs. The problems were often severe enough to make it hard to imagine children mastering the operation of the programs without the assistance of an adult. Besides making them hard to use, the difficulties clearly interfered with the programs' instructional value. In fact, the children who were able to operate the programs were also the ones who already tended to have competence in the subjects the programs were designed to teach. In line with this finding, the younger children clearly had more difficulties operating the programs; however, there were certain features of the interface that were disruptive to the older children, but not to the younger ones.

Specific Misconceptions Shown

Three examples of children's difficulties with the programs are given below. They have been selected to illustrate the range of problems the children encountered.

The return key. Difficulties with the return key provide a very clear example of a set of problems having to do with the operation of the the program itself. Virtually all of the children had trouble using the return key both in entering their answers to problems and, to a lesser extent, in making the selection for the next problem. The difficulty usually took the form of the subjects typing in their answers, but then omitting to press the return key. In this event, they would often simply sit waiting for the computer to respond. Typically the error could not easily be overcome. In spite of the experimenter repeatedly reminding the child to press return, almost half of the 5-year olds did not come to press the return without help in the entire session, even though the majority were able to key in their actual answers on their own (see Table 1).

	5-yr olds	7-yr olds
Needed help on at least one trial:		
Keying in answer:	17 (85%)	0
Using return to enter answer:	18 (90%)	6 (30%)
Using return for next problem:	17 (85%)	6 (30%)
Needed help on majority of trials:		
Keying in answer:	3 (15%)	0
Using return to enter answer:	9 (45%)	0
Using return for next problem:	8 (40%)	0

Table 1: Numbers of Children (out of 20) at Each Age Needing Help on One Trial and on the Majority of Fifteen Trials

Operating the return key may have been a problem because it served two functions: entering answers and selecting the next problem. However, mistakes similar to ones we observed have been found for adults who had to use the return key to give commands to a text editor (Norman, 1982). Probes of our subjects indicated that they did not understand why it was necessary to press the return key. They assumed that the computer "had gotten" their answer because their numbers had appeared on the screen or the menu arrow had been moved. Pressing the return made no immediate change on the screen, although it eventually, of course, produced an action on the part of the program.

Instructional sequences. The parts of the programs meant to instruct generated some problems that went beyond the simple operation of the software. The clearest example involves the remedial sequence presented after incorrect answers. Specifically, *only* 4 of the 20 5-year olds and 7 of the 20 7-year olds realized that this sequence was depicting the correct answer to the problem (see Table 2).

One possible explanation of this problem is that it was a poor design choice to use crossed lines to mark objects as they were being counted—since x's are usually associated with mistakes—but the children did not comment to this effect. They seemed just not to associate this part of the program with a remediation routine.

	5-yr olds	7-yr olds
Knew what wrong sequence meant:	4 (20%)	7 (35%)

Table 2: Numbers of Children (out of 20) at Each Age Recognizing What the Wrong Sequence Signified

Miscommunications. A further class of difficulties arose that can only be seen as misunderstandings between the child and the intent of a particular program. A striking instance of this sort occurred with several of the younger children and the addition program. These children were able to count through all of the objects in both boxes on the screen during an addition problem. They were also able to key in the total, so if they had counted correctly, they arrived at the correct answer to the problem. Our probing showed, however, that 17 out of the 20 5-year olds could not read the addition problem that had originally been set on the screen (see Table 3). They had an especially difficult time recognizing the '+' sign.

	5-yr olds	7-yr olds
Were unable to read on the screen:		
The numbers in the problem:	17 (85%)	0
The plus sign in the problem:	18 (90%)	6 (30%)
The "How many?" prompt:	17 (85%)	6 (30%)

Table 3: Numbers of Children (out of 20) at Each Age Unable to Read Relevant Parts of the Screen

It appears that when a number of identical small objects are drawn on the screen young children will count them. In this case, the program seems to be giving practice on addition, but actually these younger children were not even aware that they were answering an addition problem; they just counted. This effect first became apparent with the subtraction program. In this program, the young children were even less likely to be able to read the minus sign. Except here, of course, simply counting the objects on the screen always led to an incorrect answer.

The Interface and Instructional Effectiveness

Difficulties with the interface clearly interfered with the instructional effectiveness of the software. Nearly half of the 5-year olds (Table 1) needed help throughout the session just to operate the programs. Obviously, without that help, these children would never have come in contact with the instructional content of the programs. Similarly, some of the other 5- and 7-year olds who only needed help at the beginning of the session would probably not have mastered the operation of the programs if help had not been available at the outset.

The results also revealed that the children who were best at arithmetic were the one who had the fewest difficulties with the operation of the program. So, for example, it was found that the children's score on the arithmetic subscale of the Wechsler was negatively correlated ($r's = -.50's, p's < .05$) with the number of prompts they had to be given. It might be that these children knew how to operate the programs, so their arithmetic improved compared to the others, but no such improvement in performance during the session was found. The pattern of results is consistent the possibility that the children who were good at arithmetic had one less thing to worry about when they were trying to learn to use the programs.

Finally, there was some evidence that the programs at times directly interfered with some children's understanding of arithmetic. For 6 of the 20 5-year olds, there were occasions when they spontaneously showed (by speaking aloud or pointing) they knew the answer to a problem and yet were unable to convey that answer to the program. When the program did not respond correctly, they, more often than not, changed their answer because they now thought it was wrong.

Interaction with General Skill and/or Developmental Level

There were obviously very large age differences in response to the interface to the programs. The 5-year olds required many more instances of direct prompting ($t's > 3.59, p's < .05$) to overcome difficulties with the operation of the software. They were also simply less successful than the 7-year olds in operating the programs. Yet, we discovered a few aspects of the interface that were worse for the 7-year olds than the 5-year olds. For example, a probe question showed that the 7-year olds consistently found the pace of the programs to be boring, while the 5-year olds did not ($t = 2.49, p < .05$). As a consequence, it was not

unusual to observe that the 7-years would stop attending to the program when they had to wait for it to respond.

The age differences found must be interpreted in light of the fact that the children in the study only differed in age by two years and that the software was intended for children of these ages. With larger age differences, of course, the problem would be even worse and an interface that did not make more of a provision for developmental level would stand to generate an even greater number of difficulties.

Conclusion and Future Directions

The results of these studies show very clearly that the interface to a program—how it presents itself to a young user—is an essential consideration in the design of educational software. The design decisions made in these programs were not unreasonable and were well-intentioned. However, at critical places they violated the criteria for educational interfaces identified in the Introduction:

- *the interface must provide an entry to the content domain rather than vice versa,*

In other types of software, the user typically knows what application a program is meant to do and can use that knowledge to decipher the interface. Users of educational software will not have a similar advantage. They will not have a good understanding of the domain being taught and so will not have that entry to the interface. In our studies of software to teach addition, for example, our subjects appeared to need to know how to add already in order to be able to master the operation of the program.

- *the interface must be sensitive to the child's developmental level.*

The importance of this point became apparent several times in our studies. For example, we found that our younger students had more difficulty than the older ones in using the return key to enter their arithmetic answers and operate the program, while the older subjects objected more to the slow pace of the software. Given that the children only differed in age by two years, it is obvious the interface must make some allowance for development if it is to be effective.

In sum, then, our research has begun to show some of the wide variety of problems children have using common educational software. We found some simple, but persistent problems, like the difficulties associated with the return key. There were also instances of deep misunderstandings where the children did not recognize the point of a major component of a program or the point of the program itself. We found that these difficulties did interfere with instructional aspects of the programs. They indicate

that interface issues must be squarely faced if educational software is to be effective.

Facing those issues means studying users of educational software. We believe it also means studying how people learn in particular domains and deriving implications for interface design from that information. Arithmetic is a promising domain in this regard. A great deal is known about how children begin to learn this subject (e.g., Fuson and Hall, 1983). Some of these findings (Shannon, 1978) indicate that a "direct manipulation style" interface (Shneiderman, 1983) might be particularly well suited to teaching arithmetic. In the future, we hope to see if such an interface might eliminate a significant number of the difficulties we observed in the present study.

References

- Anderson, J.
Cognitive Psychology and Intelligent Tutoring.
In *Proceedings of the Sixth Annual Conference of the Cognitive Sciences*. Boulder, Colorado, 1984.
- Anderson, J., Boyle, C., and Yost, G.
The Geometry Tutor.
In *Proceedings of the Ninth International Joint Conference on Artificial Intelligence, Volume 1*, 1985.
- Farrell, R., Anderson, J., and Reiser, B.
Interactive Student Modelling in a Computer-based Lisp Tutor.
In *Proceedings of the Sixth Annual Conference of the Cognitive Sciences*. Boulder, Colorado, 1984.
- Fuson, K. and Hall, J.
The Acquisition of Early Number Word Meanings: A Conceptual Analysis and Review.
In *The Development of Mathematical Thinking*, H. Ginsburg, (Ed.), Academic Press, New York, 1983.
- Pirolli, P.
A Cognitive Model and Computer Tutor for Programming Recursion.
Manuscript submitted for publication, 1986.
- Norman, D.
Learning and Memory.
Freeman, San Francisco, 1982.
- Shannon, L.
Spatial Strategies in the Counting of Young Children.
Child Development 49:1212-1213, 1978.
- Shneiderman, B.
Direct Manipulation: A Step Beyond Programming Languages.
IEEE Computer 16:57-69, 1983.